



DuocUC[®] 

Colecciones

DSY1102 - Desarrollo Orientado a Objetos



Contenido

01

¿Qué es el Framework de Colecciones?

02

La interfaz List

03

La clase ArrayList

04

Interfaz Map

05

Comparación Visual

06

Ejemplo practico
List<Animal>

07

Ejemplo practico
ArrayList<Animal>

08

Map en practica Operaciones

09

Recorriendo Colecciones

10

Resumen de métodos
Comunes

11

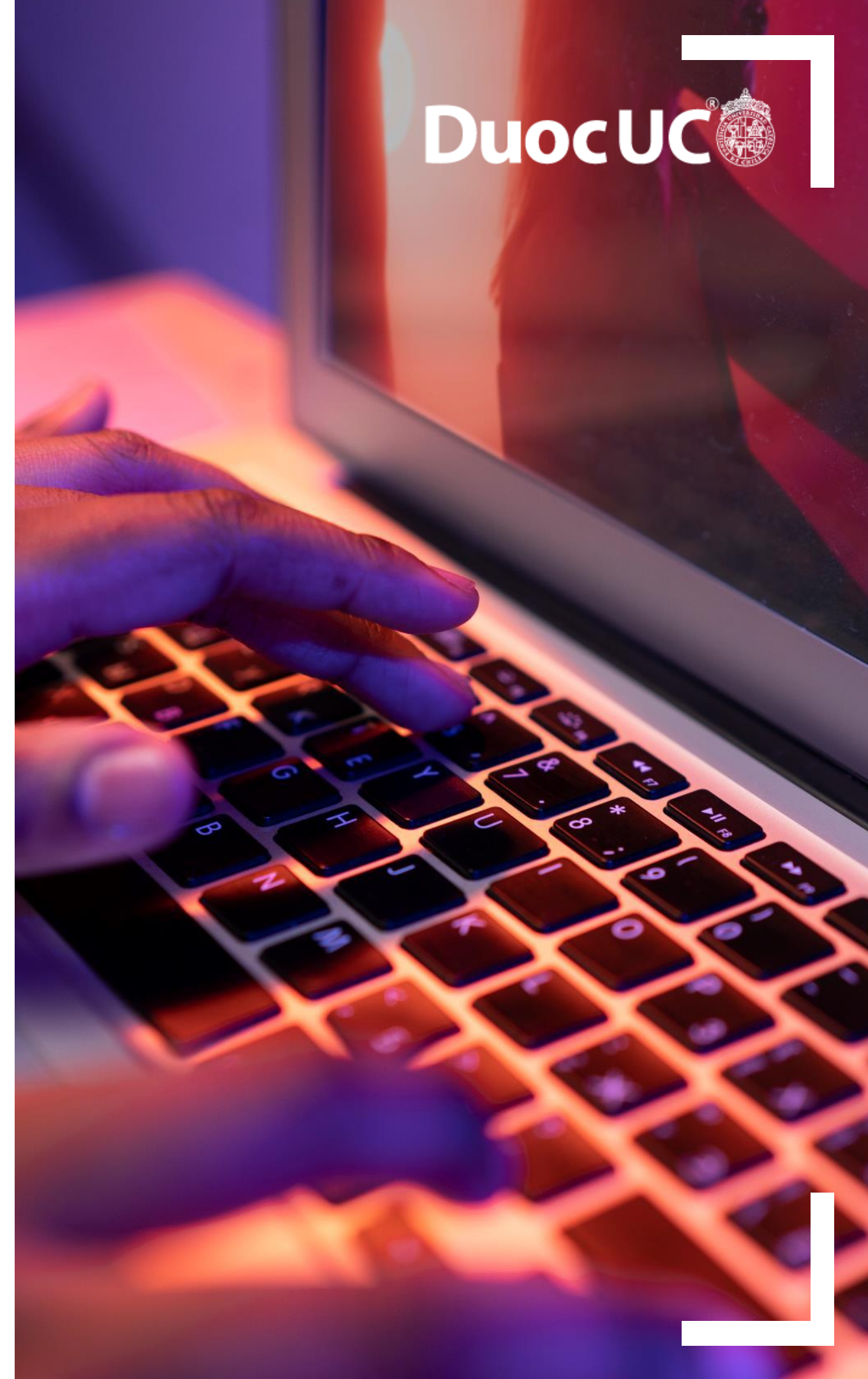
Ejemplo practico
Colecciones en
ZooApp

12

Buenas practicas y errores
comunes

13

Conclusiones y Próximos pasos



```
List<Animal> zoo = new ArrayList<>(); zoo.add(new Perro()); zoo.add(new Gato()); Map<String, Animal> mapa = new HashMap<>(); mapa.put("A1", new Pato());
```



Programación Orientada a Objetos

Java

Colecciones en Java

List, ArrayList y Map con ejemplos prácticos basados en el ZooApp (Perro, Gato, Pato).



Estructuras de Lista
List & ArrayList



Diccionarios
Map & HashMap

 ZooApp.java

```
// 1. Usando List y ArrayList
List<Animal> listaAnimales = new ArrayList<>();
listaAnimales.add(new Perro());
listaAnimales.add(new Gato());

// 2. Usando Map (Clave - Valor)
Map<String, Animal> mapaZoo = new HashMap<>();
mapaZoo.put("A001", new Pato());
mapaZoo.put("A002", new Perro());

// Recuperar un valor
Animal miPato = mapaZoo.get("A001");
```



2026



15 slides



¿Qué es el Framework de Colecciones?

Estructuras de datos dinámicas en Java

💡 Concepto Central

El **Java Collections Framework** provee una arquitectura unificada para almacenar y manipular grupos de objetos, como listas de animales en un zoológico.



Tamaño dinámico

Crecen o se reducen automáticamente, a diferencia de los arreglos.



Múltiples estructuras

Listas, conjuntos y mapas según la necesidad del problema.



Operaciones listas

Métodos predefinidos para buscar, ordenar e iterar datos.

En resumen:

- 📌 Agrupan múltiples elementos en una sola unidad.

Flujo de Colecciones





La Interfaz List

Colecciones ordenadas con acceso por índice

★ Características Principales



Colección Ordenada

Mantiene el orden exacto en el que se insertan los elementos. El primer elemento está en la posición inicial.



Permite Duplicados

Puedes almacenar el mismo objeto o valores idénticos múltiples veces en diferentes posiciones.



Acceso por Índice

Los elementos se acceden de forma precisa mediante un índice numérico basado en cero (0, 1, 2...).



Implementaciones comunes

ArrayList: Rápida lectura, array dinámico.

LinkedList: Rápida inserción/borrado, nodos enlazados.



Estructura Visual con Animales



Ejemplo Conceptual

```
// Declaración usando la interfaz List
List<Animal> animales = new ArrayList<>();

// Agregar manteniendo el orden
animales.add(new Perro()); // Índice 0
animales.add(new Gato()); // Índice 1

// Acceso posicional
Animal primero = animales.get(0);
```



Colecciones



Interfaz List



La Clase ArrayList

Listas dinámicas y redimensionables

Operaciones

ArrayList es una implementación de List basada en un arreglo que crece automáticamente según sea necesario.



add(E e)

Añade un elemento al final
lista.add(new Perro())



get(int index)

Obtiene elemento por índice
Animal a = lista.get(0)



remove(int index)

Elimina el elemento
lista.remove(1)



size()

Retorna cantidad de elementos
int total = lista.size()

Crecimiento Dinámico

1. `lista.add("Perro");`



2. `lista.add("Gato");`



3. `lista.add("Pato");` // Redimensiona si es necesario



Interfaz Map

Almacenamiento de pares Clave-Valor

¿Qué es un Map?

Una colección que mapea **claves** a **valores**. No hereda de Collection.



Claves Únicas

Un Map no puede contener claves duplicadas. Cada clave mapea a un único valor.



Acceso por Key

Se accede a los elementos proporcionando su clave (ej. un ID o nombre), no por un índice.



Búsqueda Rápida

Ideal para buscar información rápidamente cuando se conoce el identificador (ej. HashMap).

Diccionario de Animales



ZooRegistry.java

```
Map<String, Animal> zoo = newHashMap<>();

// Añadir pares Clave-Valor
zoo.put("perro1", newPerro("Max"));
zoo.put("gato1", newGato("Luna"));

// Recuperar un valor usando su Clave
Animal miAnimal = zoo.get("perro1");
```



Comparación Visual

List vs ArrayList vs Map

⌵ Diferencias Clave

Elige la estructura correcta según cómo necesites organizar y acceder a tus animales.

≡ List (Interfaz)

Contrato general para colecciones ordenadas. Admite duplicados.

☞ ArrayList (Clase)

Implementación rápida de List usando arrays dinámicos.

🔑 Map (Interfaz)

Almacena pares Clave-Valor. Claves únicas, acceso rápido.

Regla de oro:

List para secuencias. Map para buscar por un ID.

Estructura y Caso de Uso

List/ArrayList

- ↓ Ordenado
- 📄 Admite Duplicados

Secuencia de Animales

0 🐶 Perro 1 🐱 Gato 2 🦆 Pato

`animales.get(1) %Gato`

Map

- 🔀 Sin orden
- 🔑 Claves Únicas

Diccionario ID Animal

"A1" 🐶 Perro "A2" 🐱 Gato

`mapa.get("A1") %Perro`



Uso típico en ZooApp: Usa List para mostrar todos los animales en una tabla. Usa Map para encontrar rápidamente un animal escaneando su código de collar.



Ejemplo Práctico: List<Animal>

Almacenando y recorriendo animales en secuencia

? ¿Cuándo usar List?

List es ideal cuando necesitas mantener el orden exacto de inserción y permitir elementos repetidos en tu colección.



Orden garantizado

El primer animal agregado se mantiene siempre en la primera posición.



Acceso por índice

Puedes obtener cualquier animal rápidamente usando su posición (0, 1, 2...).



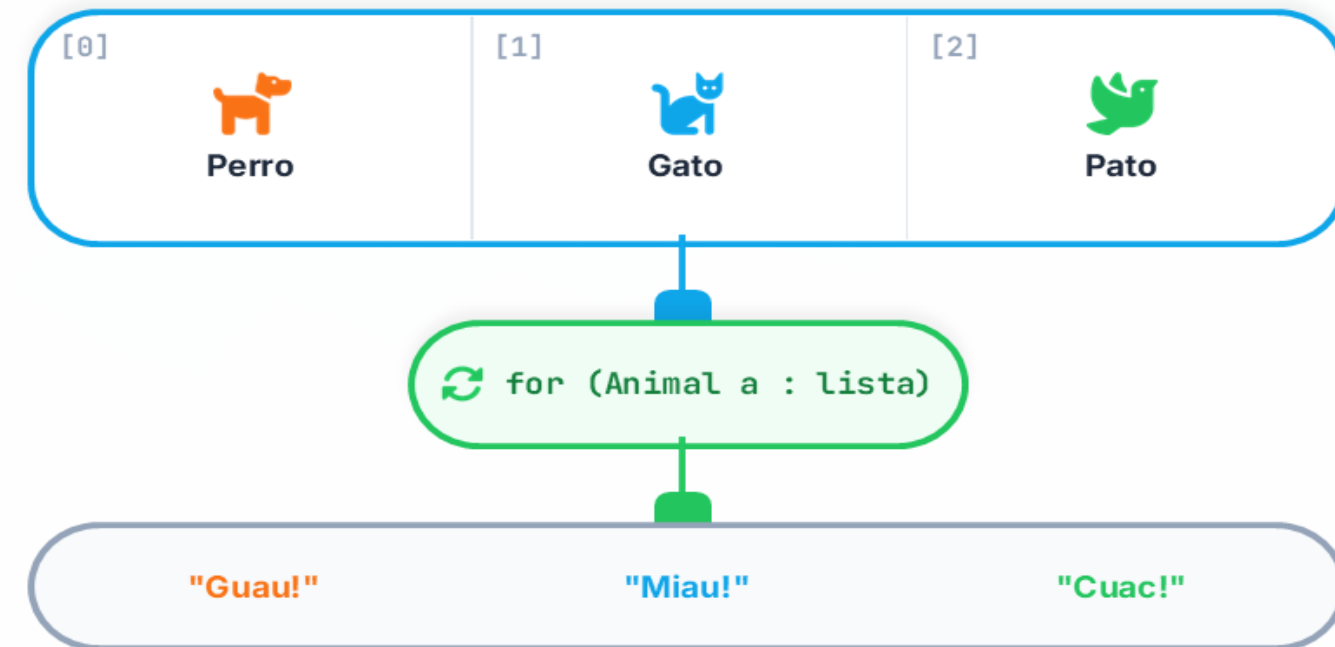
Permite duplicados

Es posible registrar al mismo animal varias veces si es necesario.

Importante:

 Utiliza un bucle for-each para recorrerlas

Flujo de Almacenamiento y Recorrido



// Ejemplo de List y for-each

```
List<Animal> animales = new ArrayList<>();
animales.add(new Perro("Rex")); // Índice [0]
animales.add(new Gato("Miau")); // Índice [1]
animales.add(new Pato("Donald")); // Índice [2]

for (Animal a : animales) { a.hacerSonido(); }
```



Ejemplo Práctico: ArrayList<Animal>

Flexibilidad y tamaño dinámico en acción

Operaciones Comunes

ArrayList crece dinámicamente. Veamos cómo gestionar una lista de animales paso a paso.



Agregar (add)

Añade elementos al final o en un índice específico.



Obtener (get)

Acceso rápido por índice (O(1)).



Eliminar (remove)

Elimina por objeto o índice; reorganiza la lista.

Ideal para: Lecturas frecuentes y tamaño variable.

Flujo de Operaciones

1. add()



Perro (0)
Gato (1)

2. get(1)



Retorna Gato

3. remove(0)



Gato pasa al índice 0

Código ArrayList en Acción

```
// 1. Crear e inicializar
List<Animal> zoo = new ArrayList<>();

// 2. Agregar elementos (add)
zoo.add(new Perro()); // Índice 0
zoo.add(new Gato()); // Índice 1

// 3. Obtener (get)
Animal a = zoo.get(1); // Retorna el Gato

// 4. Eliminar (remove)
zoo.remove(0); // Elimina Perro, Gato pasa a ser 0
```



Map en Práctica: Operaciones

Uso de put, get, containsKey y remove con el catálogo del zoológico

Métodos Principales

Un **Map** asocia cada clave única con un valor, funcionando como un diccionario. Es la estructura ideal para búsquedas rápidas mediante un identificador.

Método	Descripción
<code>put(k, v)</code>	Añade o actualiza el par clave-valor.
<code>get(k)</code>	Devuelve el valor de la clave especificada.
<code>containsKey(k)</code>	Retorna true si la clave ya existe.
<code>remove(k)</code>	Elimina el par asociado a la clave.

Regla de Oro:
💡 Las claves no pueden duplicarse en un Map.

Catálogo del Zoológico

CLAVE (String ID)

"P-01"

"G-02"

"A-03"

VALOR (Animal)

 Perro Max

 Gato Luna

 Pato Donald

🔗

🔗

🔗

🔍 Búsqueda rápida:

`catalogo.get("G-02")` devuelve a Luna

```
Map<String, Animal> zoo = new HashMap<>();
zoo.put("P-01", new Perro("Max"));
Animal a = zoo.get("P-01"); // Devuelve a Max
boolean existe = zoo.containsKey("G-02"); // false
```



Recorriendo Colecciones

for-each en List vs entrySet en Map con ejemplos de animales

Formas de Iterar

Recorrer colecciones es esencial para procesar los datos de nuestros animales y ejecutar operaciones sobre ellos.

for-each en List

La forma más directa y legible de recorrer elementos secuencialmente uno por uno.

entrySet() en Map

Permite iterar accediendo tanto a la clave (identificador) como al valor (animal) simultáneamente.

Alternativas en Map

Usa **keySet()** si solo necesitas claves, o **values()** si solo necesitas los objetos animal.

Ejemplos Visuales y Código

Iterando un ArrayList (Secuencia)



```
for ( Animal a : listaAnimales) { System.out.    println ("Animal: " + a.  getNombre ()); }
```

Iterando un Map con entrySet() (Clave-Valor)



```
for ( Map.Entry <String , Animal > e : mapa.  entrySet ()) {  
    System.out.println(e.getKey() + " -> " + e.  getValue ());  
}
```




Resumen de Métodos Comunes

Operaciones esenciales para List, ArrayList y Map usando animales

List / ArrayList

+ add(E e)

Añade un elemento al final de la lista.

```
lista.add(pato);
```

👉 get(int index)

Obtiene el elemento en una posición específica.

```
Animal a = lista.get(0);
```

🗑 remove(int index)

Elimina el elemento de la posición indicada.

```
lista.remove(1); // Adiós gato
```

📏 size()

Devuelve la cantidad actual de elementos.

```
int total = lista.size();
```

Map (Clave - Valor)

Búsquedas rápidas

➡ put(K, V)

Inserta un nuevo par o actualiza el valor de una clave existente.

```
mapa.put("ID1", perro);
```

❓ containsKey(K)

Verifica si el mapa contiene una clave específica (retorna boolean).

```
if (mapa.containsKey("ID2"))
```

🔍 get(K)

Obtiene el valor asociado a la clave. Retorna null si no existe.

```
Animal a = mapa.get("ID1");
```

🔗 remove(K)

Elimina el par asociado a la clave y devuelve su valor previo.

```
mapa.remove("ID1");
```



Ejemplo Práctico: Colecciones en ZooApp

Uso combinado de ArrayList y Map en un zoológico

Arquitectura de Datos

ArrayList para la vista

Almacena los animales en orden de registro para mostrarlos en la UI o recorrerlos fácilmente.

```
List<Animal> listaAnimales;
```

Map para búsqueda

Indexa cada animal por su identificador único (ID) para acceso instantáneo ($O(1)$).

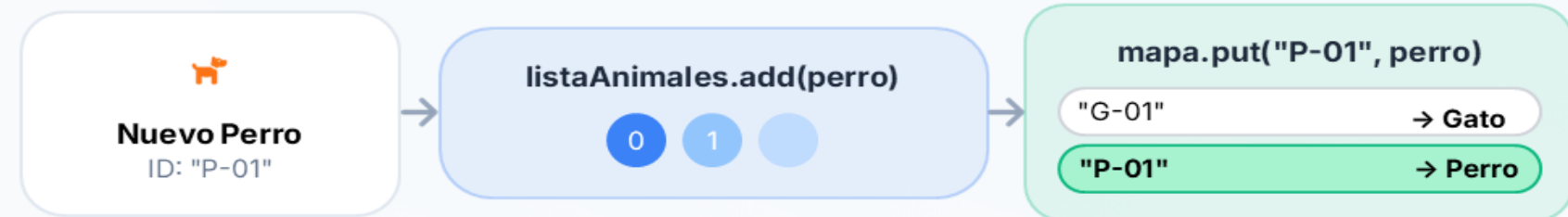
```
Map<String, Animal> indiceAnimales;
```

Sincronización

Al registrar un animal, se añade a **ambas** colecciones para mantener la consistencia.

Flujo de Datos: Registro y Consulta

1. Flujo de Registro



2. Flujos de Consulta

Mostrar todos (List)

```
for(Animal a : listaAnimales) {  
    mostrar(a);  
}
```



Buscar específico (Map)

```
Animal a = mapa.get("P-01");  
if(a != null) mostrarDetalle(a);
```

"P-01" → 

Buenas Prácticas y Errores Comunes

Uso eficiente de Colecciones en Java

⚠ Errores a Evitar



Confundir Índice y Clave

List usa índice, Map usa clave



Acceso sin validar

No revisar containsKey() o size()



Olvidar Null checks

Permitir nulls puede romper lógica



Acoplamiento fuerte

Declarar ArrayList en lugar de List



Bucle ineficiente

Usar get(i) en colecciones no indexadas



Estos errores causan excepciones en runtime



Checklist de Buenas Prácticas



Programar contra Interfaces

List<Animal> = new ArrayList<>()



Elegir la colección adecuada

ArrayList para acceso rápido, Map para búsqueda



Usar For-Each

Código más limpio al recorrer



Generics

Siempre especificar el tipo (ej. <String, Animal>)



Conocer equals() y hashCode()

Clave en Map requiere correcta implementación



Código más robusto y mantenible



Ejemplo: ZooApp

Programar a la Interfaz:

```
// Bien:  
List<Animal> lista = new ArrayList<>();  
  
// Evitar:  
ArrayList<Animal> lista =  
new ArrayList<>();
```

Validación en Map:

```
if (zoologico.containsKey("A01")) {  
    Animal a = zoologico.get("A01");  
    a.alimentar();  
} else {  
    System.out.println("No encontrado");  
}
```



Resultado: Código flexible y seguro



Consejo: Evita IndexOutOfBoundsException verificando size() antes de usar get().



Conclusiones y Próximos Pasos

Cierre de Colecciones en Java

📋 Síntesis Final

El framework de **Colecciones** proporciona estructuras de datos que mejoran la eficiencia, flexibilidad y legibilidad del código al manejar grupos de objetos como animales.

📁 **Organización**
Estructuras estandarizadas para agrupar datos.

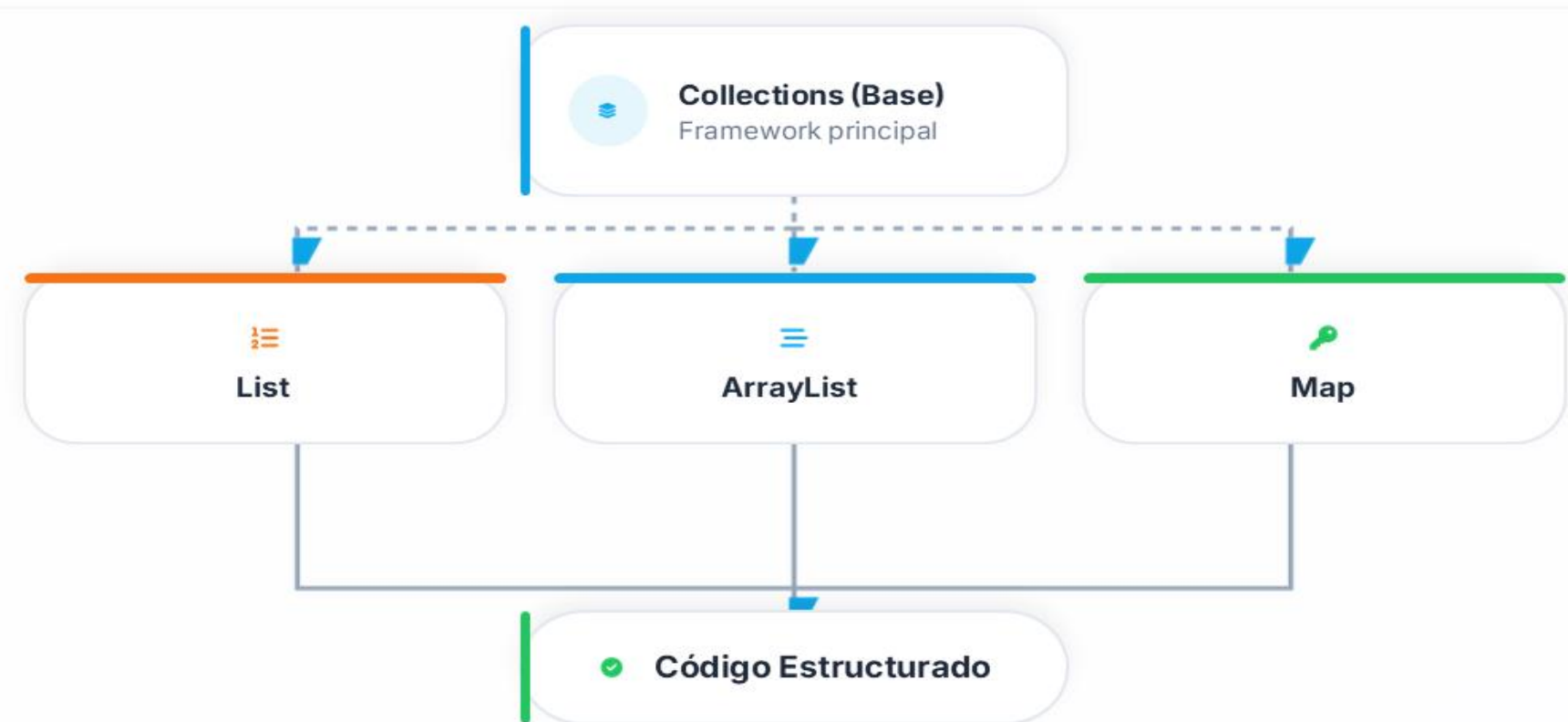
⚡ **Acceso Eficiente**
Rendimiento óptimo para búsquedas e inserciones.

📄 **Código Mantenable**
Interfaces comunes que facilitan la reutilización.

Resultado:

📋 Datos mejor estructurados y reutilizables

Visión General de Colecciones



→ Próximos Pasos

1 Practicar Set y únicos

2 Usar Iteradores en List

3 Aprender Streams API

Felicitaciones
¡Has finalizado el módulo!

DuocUC[®]



CERCANÍA. LIDERAZGO. FUTURO.

7
AÑOS
ACREDITADO

 Comisión Nacional
de Acreditación
CNA-Chile

NIVEL DE EXCELENCIA
HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la
calidad / Vinculación con el Medio / Investigación, creación y/o innovación